Hitec
07 de mayo de 2025

Máquinas de Estado en Sistemas Embebidos

Federico Nuñez Frau - HITEC S.R.L.

HITEC SRL
sensing & control

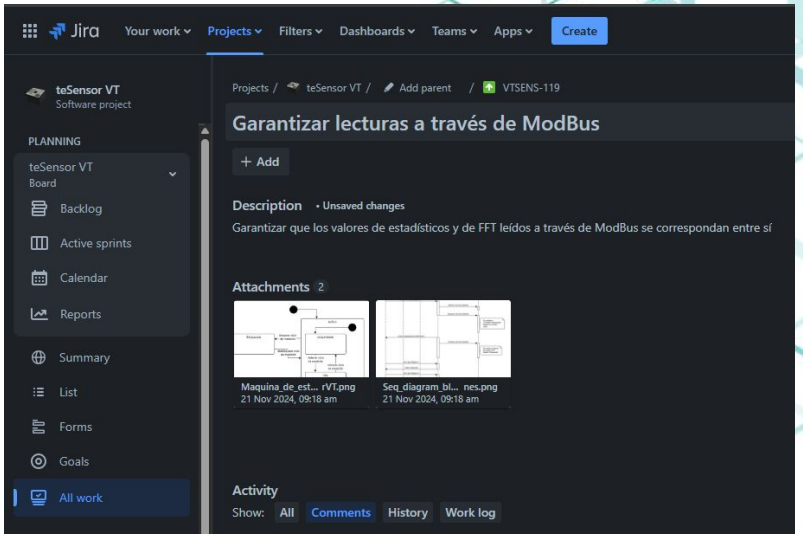
HITEC SRL
sensing & control

Contenidos

1. Motivación
2. Máquinas de Estado
3. *Active Objects*
4. Ejemplo Sencillo
5. Implementación en C
6. Herramienta: *QP Framework*
7. Resumen
8. Trabajo Futuro

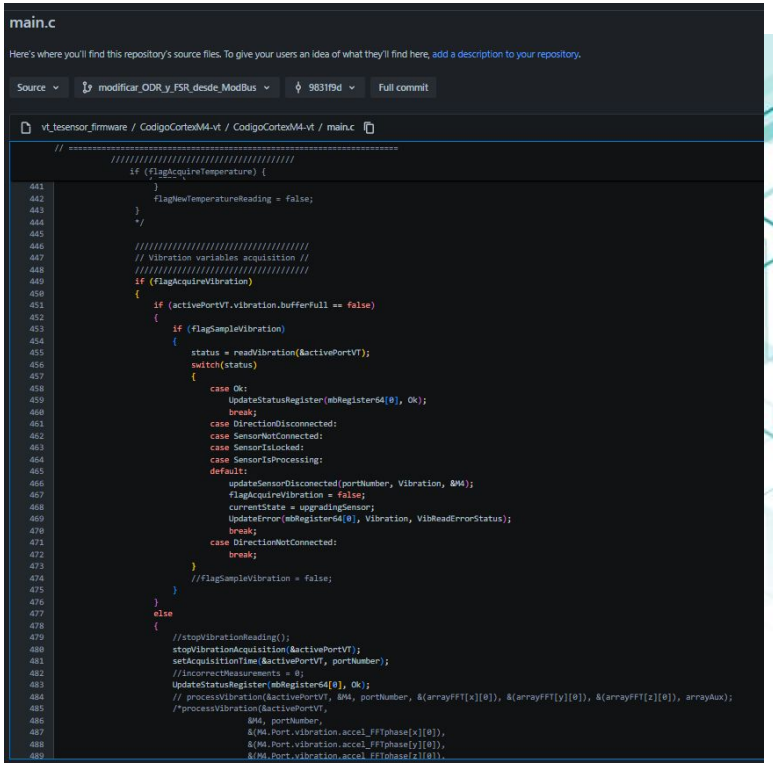
1. Motivación

- **Agregado** de una **nueva funcionalidad** al teSensorVT.
- Es decir, construir sobre lo que ya está construido.
- La funcionalidad es sencilla y clara.



1. Motivación

- Es más fácil y rápido construir sobre algo estable y claro.
- Capturas del commit: 08/11/2024



1. Motivación

- Es más fácil y rápido construir sobre algo estable y claro.
- Capturas del commit: 08/11/2024

main.c

Here's where you'll find this repository's source files. To give your users an idea of what they'll find here, add a description to your repository.

Source | modificar_ODR_y_FSR_desde_ModBus | 9831f9d | Full commit

vt_tesensor_firmware / CodigoCortexM4-vt / CodigoCortexM4-vt / main.c

```
        // ==================================================================
                    ////////////////////////////////////////
                        if (flagAcquireTemperature) {
441                         }
442                         flagNewTemperatureReading = false;
443                     }
444                     */
445
446                     ////////////////////////////////////////
447                     // Vibration variables acquisition //
448                     ////////////////////////////////////////
449                     if (flagAcquireVibration)
450                     {
451                         if (activePortVT.vibration.bufferFull == false)
452                         {
453                             if (flagSampleVibration)
454                             {
455                                 status = readVibration(&activePortVT);
456                                 switch(status)
457                                 {
458                                     case Ok:
459                                         UpdateStatusRegister(mbRegister64[0], Ok);
460                                         break;
461                                     case DirectionDisconnected:
462                                     case SensorNotConnected:
463                                     case SensorIsLocked:
464                                     case SensorIsProcessing:
465                                     default:
466                                         updateSensorDisconected(portNumber, Vibration, &M4);
467                                         flagAcquireVibration = false;
468                                         currentState = upgradingSensor;
469                                         UpdateError(mbRegister64[0], Vibration, VibReadErrorStatus);
470                                         break;
471                                     case DirectionNotConnected:
472                                         break;
473                                 }
474                                 //flagSampleVibration = false;
475                             }
476                         }
477                         else
478                         {
479                             //stopVibrationReading();
480                             stopVibrationAcquisition(&activePortVT);
481                             setAcquisitionTime(&activePortVT, portNumber);
482                             //incorrectMeasurements = 0;
483                             UpdateStatusRegister(mbRegister64[0], Ok);
484                             // processVibration(&activePortVT, &M4, portNumber, &(arrayFFT[x][0]), &(arrayFFT[y][0]), &(arrayFFT[z][0]), arrayAux);
485                             /*processVibration(&activePortVT,
486                                             &M4, portNumber,
487                                             &(M4.Port.vibration.accel_FFTphase[x][0]),
488                                             &(M4.Port.vibration.accel_FFTphase[y][0]),
489                                             &(M4.Port.vibration.accel_FFTphase[z][0]),
```

1. Motivación

- Es más fácil y rápido construir sobre algo estable y claro.
- Capturas del commit: 08/11/2024
- Variables globales: **33 en total.**

```
#include "sam.h"
#include "main_cnf.h"

volatile float adcData1Result, adcData2Result, adcData3Result;

volatile uint8_t FlagNewMessageReceived,flagNewActionRequiered;
volatile uint8_t flagNewTemperatureReading,flagNewCurrentReading;
volatile uint8_t ADSRangeSelected,debugging;
volatile uint8_t adcRun;
volatile uint8_t FlagNewADC1Value, FlagNewADC2Value, FlagNewADC3Value;

uint32_t auxTimer0,auxTimer1,auxTimer2,auxTimer3,auxTimer4,auxTimer5;
volatile uint8_t flagAcquireTemperature, flagAcquireVibration, flagAcquireMagneticField;
volatile uint8_t flagSampleTemperature, flagSampleMagneticField, flagSampleVibration;

volatile uint8_t FlagNewAcquisition;
volatile uint8_t mbAddress,mbPort,mbBaudPar; // ModBus RTU variables
volatile uint32_t nCountsTimer; // Number of counts for the new measure timer

#include "main.h"
#include "modbus/RTU.h"

GPIOP_t lis2DIntPin;
GPIOP_t errSig; // Pin that drives a led to indicate there is an error
```

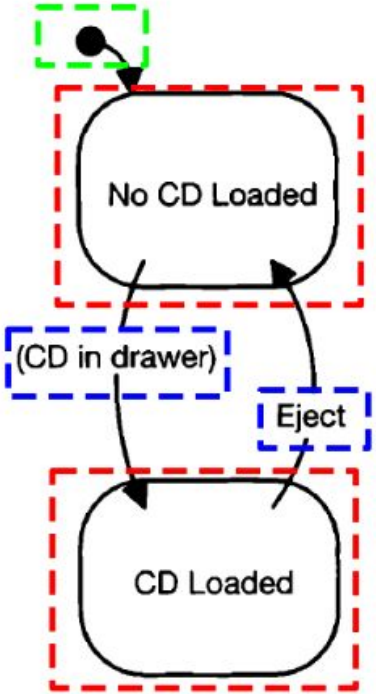
1. Motivación

- **Código entendible**:
  - Máquinas de estado para implementar lógicas complejas.
- **Extender funcionalidades → facilidad de evolución**:
  - Modularización para descorrelacionar distintas partes del código.
  - Infraestructura sólida.

2. Máquinas de Estado

- **Estados**.
- **Eventos**.
- **Inicio**.

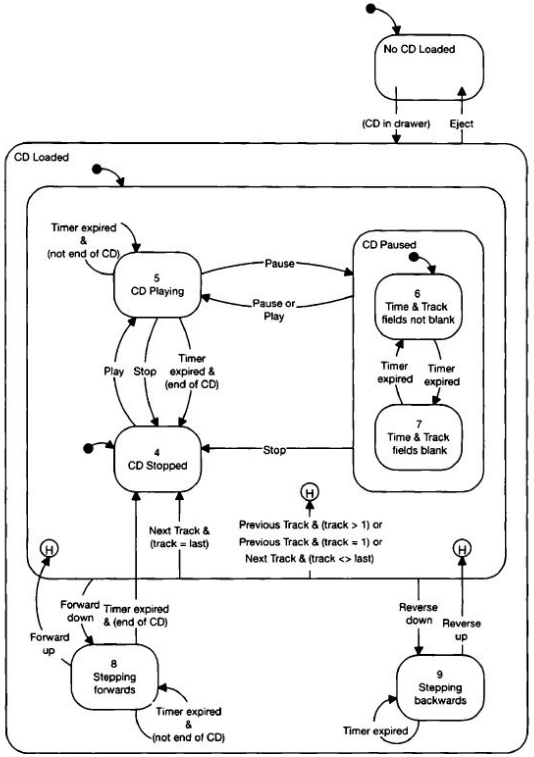
- Permiten describir lógicas complejas de manera más clara.



2. Máquinas de Estado

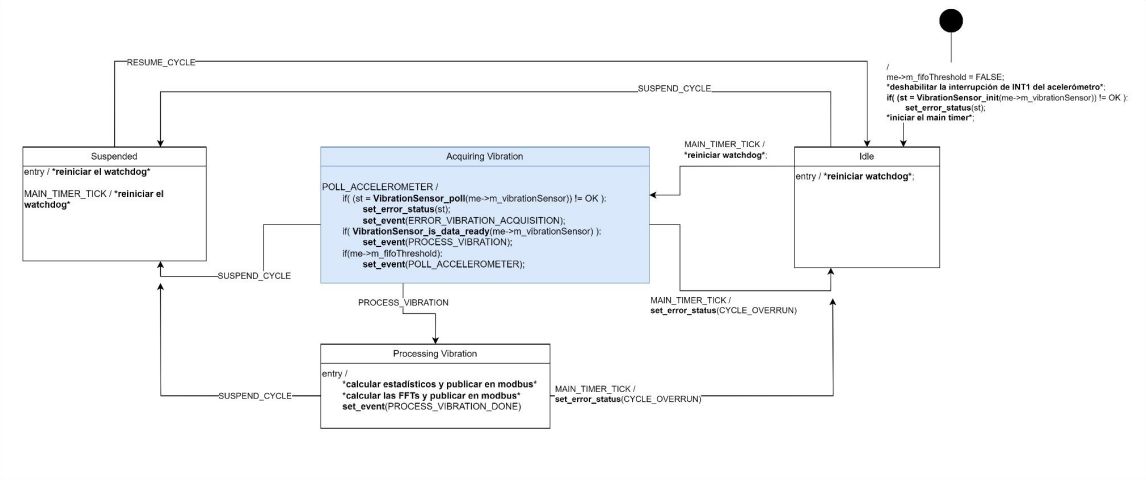
- **Estados**.
- **Eventos**.
- **Inicio**.

- Permiten describir lógicas complejas de manera más clara.



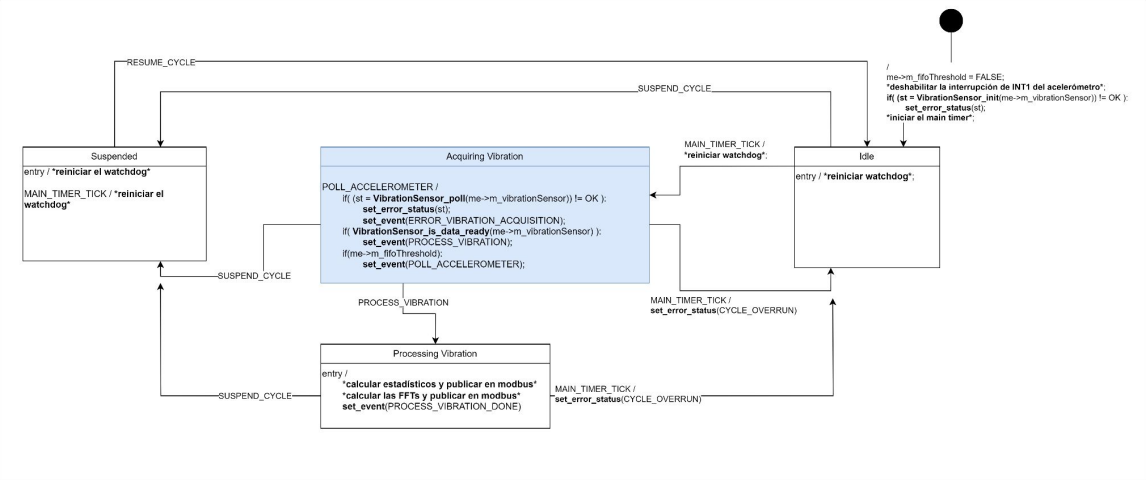
2. Máquinas de Estado

- Ejemplo teSensorVT.
- Estado inicial: Acquiring Vibration.
- Ocurren 2 eventos “al mismo tiempo”:
  - POLL_ACCELEROMETER.
  - SUSPEND_CYCLE.
- ¿Cuál es el resultado?



2. Máquinas de Estado

- Ejemplo teSensorVT.
- Estado inicial: Acquiring Vibration.
- Ocurren 2 eventos “al mismo tiempo”:
  - POLL_ACCELEROMETER.
  - SUSPEND_CYCLE.
- ¿Cuál es el resultado?



- **Rta: Depende.**

2. Máquinas de Estado

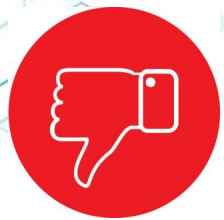
```
//...

if( pollAccelerometer == true)
{
    // .. ejecutar una acción asociada al evento
}

if( suspendCycle == true)
{
    // .. ejecutar una acción asociada al evento
}

//...
```

- Importa el **orden de ocurrencia** de los eventos.
- Si ambos eventos ocurrieron antes de ejecutar una nueva iteración de la máquina de estados → **pollear los eventos no garantiza que se procesen en el orden correcto.**

2. Máquinas de Estado

HITEC SRL
sensing & control

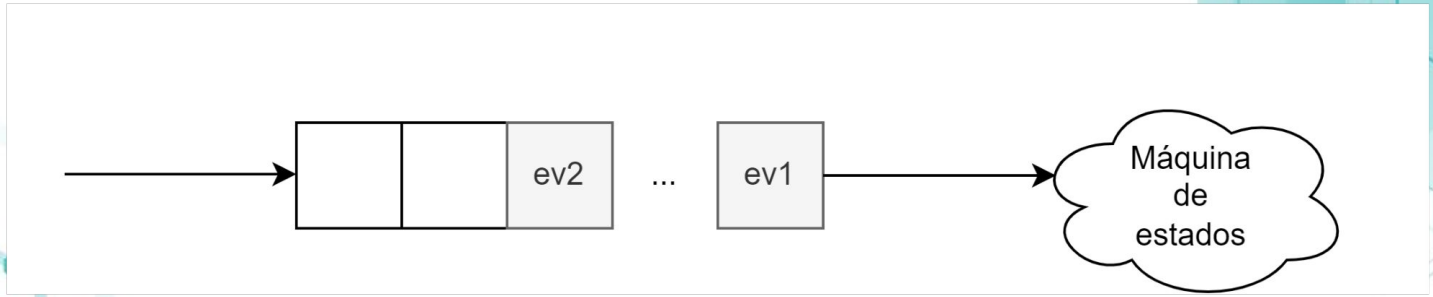
```
//...

if( pollAccelerometer == true)
{
    // .. ejecutar una acción asociada al evento
}

else if( suspendCycle == true)
{
    // .. ejecutar una acción asociada al evento
}

//...
```



- Importa el **orden de ocurrencia** de los eventos.
- Si ambos eventos ocurrieron antes de ejecutar una nueva iteración de la máquina de estados → **pollear los eventos no garantiza que se procesen en el orden correcto.**
- Qué pasa si el evento “pollAccelerometer” está siempre en true?

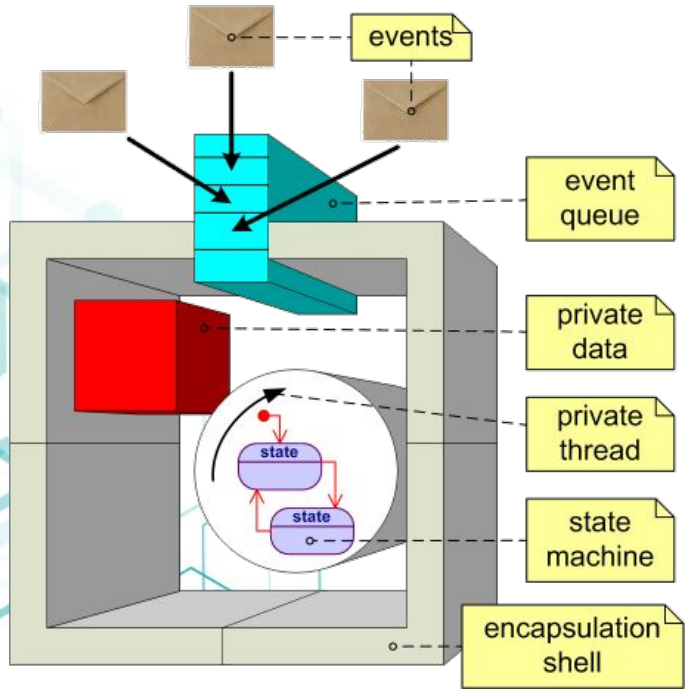
2. Máquinas de Estado

- El **primer** evento que llega es el **primero** que se procesa.



- Se puede usar una **FIFO de eventos**.
- Los **eventos** no se pollean, sino que **se encolan**.
- Luego se procesan 1 la vez, asegurando el orden de llegada a la FIFO.

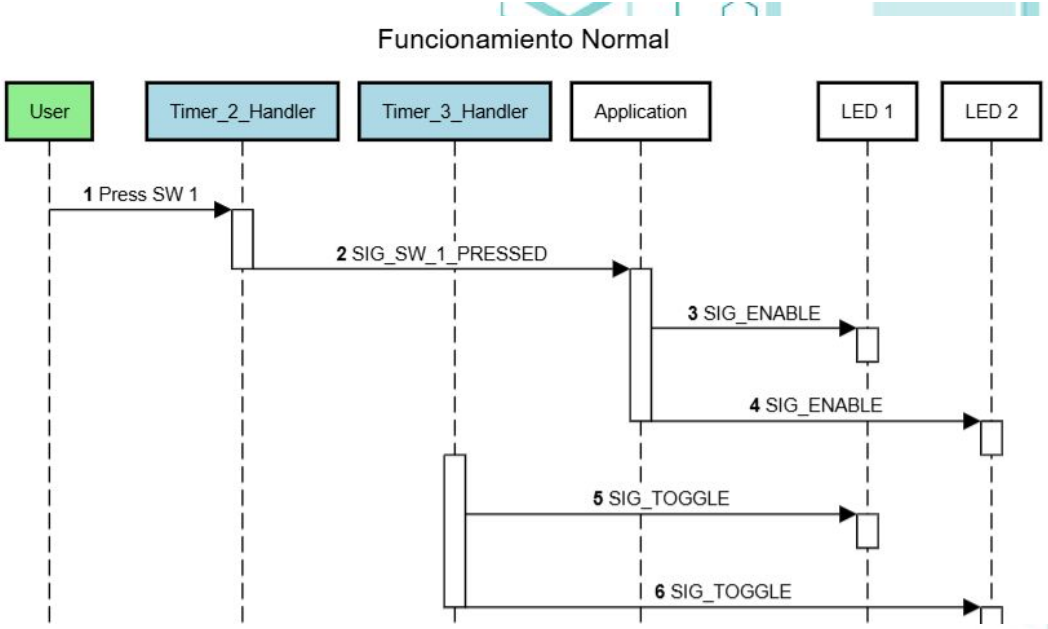
3. *Active Objects*



- Máquina de estados + FIFO de eventos + datos privados (variables) + thread de ejecución.
- La única manera de comunicarse con el AO es a través de la FIFO.
- Datos privados: NADIE puede acceder a los datos privados desde afuera.
- Thread de ejecución: Aplica para sistemas expropiativos, cada AO corre en su propio thread.

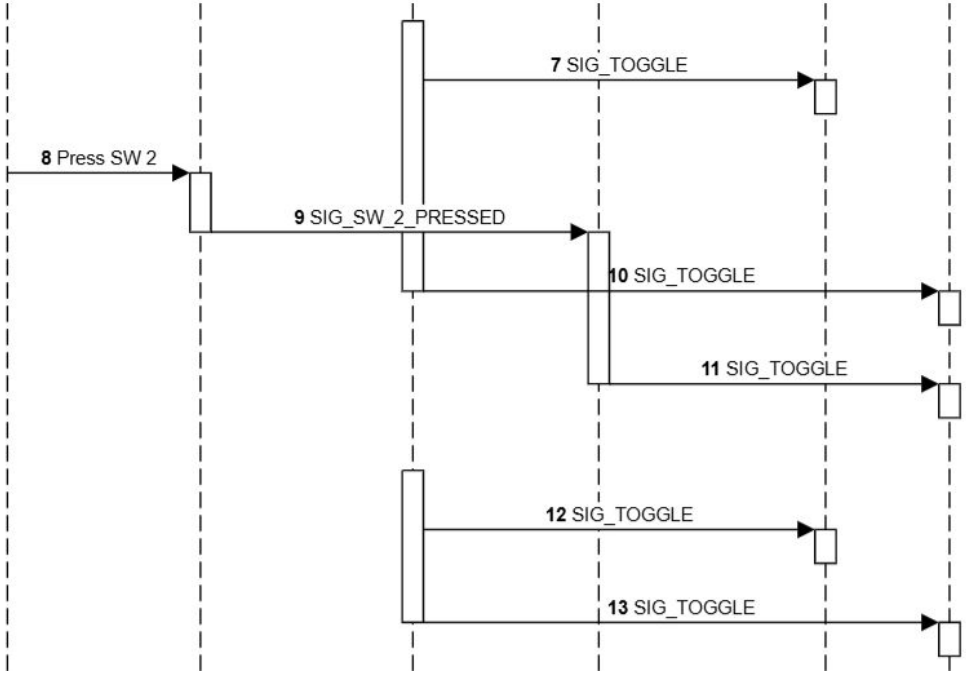
4. Ejemplo Sencillo

- Se tienen 2 LEDs que parpadean con distintas configuraciones.
- Al presionar el switch 1, ambos LEDs parpadean cada 500 ms.
- Se puede presionar el switch 2 para apagar/prender el LED 2 y alterar el parpadeo de ambos LEDs.
- Al presionar el switch 1 de nuevo, ambos LEDs se apagan hasta que se vuelva a presionar el switch 1 nuevamente.



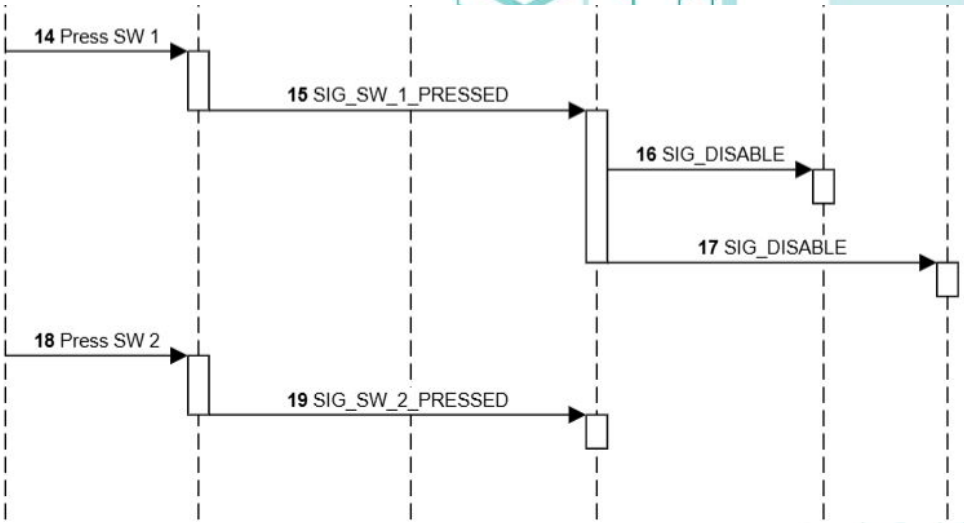
4. Ejemplo Sencillo

- Se tienen 2 LEDs que parpadean con distintas configuraciones.
- Al presionar el switch 1, ambos LEDs parpadean cada 500 ms.
- Se puede presionar el switch 2 para apagar/prender el LED 2 y alterar el parpadeo de ambos LEDs.
- Al presionar el switch 1 de nuevo, ambos LEDs se apagan hasta que se vuelva a presionar el switch 1 nuevamente.

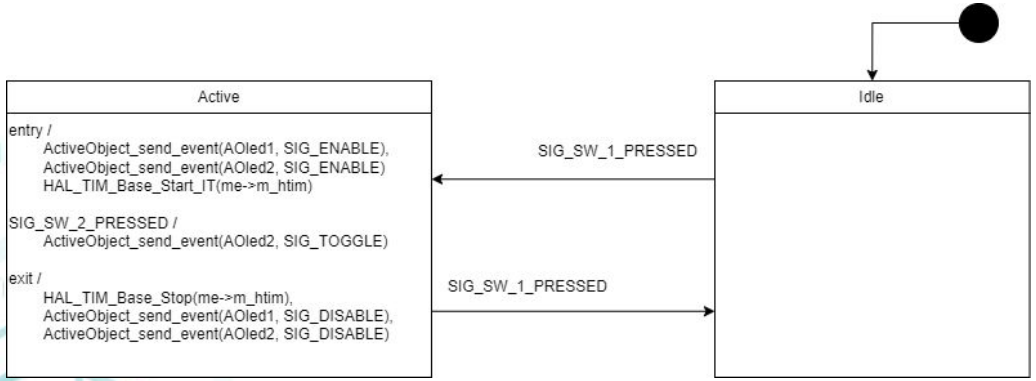


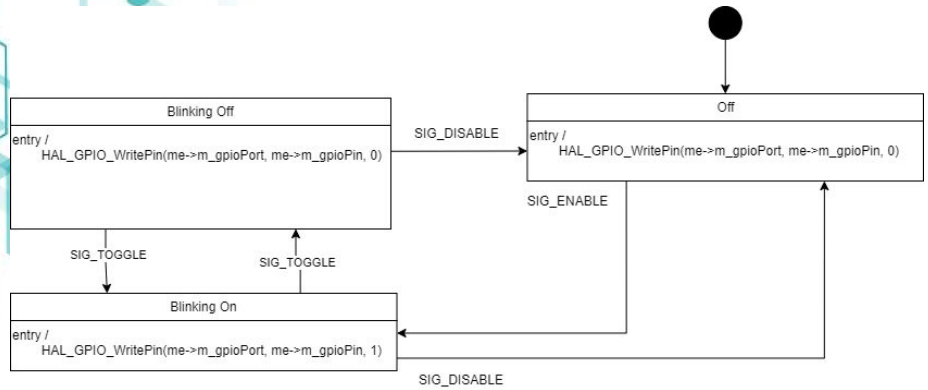
4. Ejemplo Sencillo

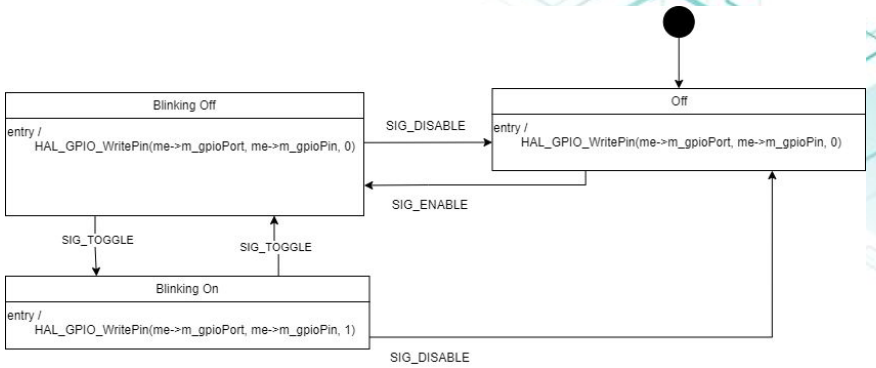
- Se tienen 2 LEDs que parpadean con distintas configuraciones.
- Al presionar el switch 1, ambos LEDs parpadean cada 500 ms.
- Se puede presionar el switch 2 para apagar/prender el LED 2 y alterar el parpadeo de ambos LEDs.
- Al presionar el switch 1 de nuevo, ambos LEDs se apagan hasta que se vuelva a presionar el switch 1 nuevamente.



4. Ejemplo Sencillo

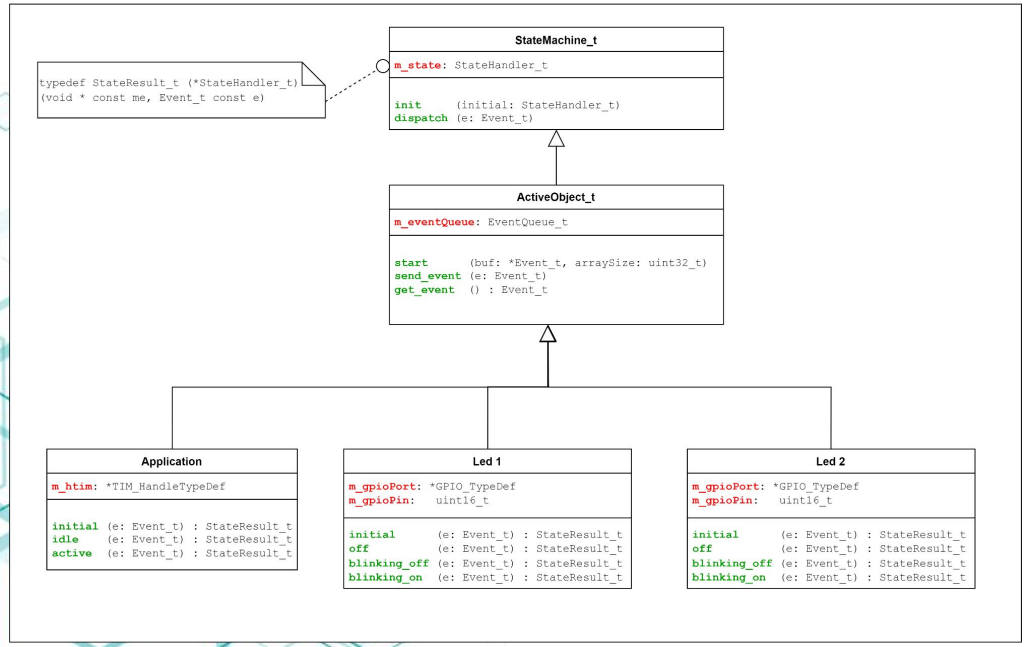






5. Implementación en C

HITEC SRL
sensing & control



WG14/N1124 Committee Draft — May 6, 2005 **ISO/IEC 9899:TC2**

12 Each non-bit-field member of a structure or union object is aligned in an implementation-defined manner appropriate to its type.

13 Within a structure object, the non-bit-field members and the units in which bit-fields reside have addresses that increase in the order in which they are declared. A pointer to a structure object, suitably converted, points to its initial member (or if that member is a bit-field, then to the unit in which it resides), and vice versa. There may be unnamed padding within a structure object, but not at its beginning.

14 The size of a union is sufficient to contain the largest of its members. The value of at most one of the members can be stored in a union object at any time. A pointer to a union object, suitably converted, points to each of its members (or if a member is a bit-field, then to the unit in which it resides), and vice versa.

- Estándar de C, sección 6.7.2.13
- Un puntero a **`Application`** puede tratarse como un puntero a **`ActiveObject_t`**.
- Lo mismo para cualquiera 2 clases padre e hijo.

5. Implementación en C

HITEC SRL
sensing & control

- ¿Cómo se envía un evento?

```c
void ActiveObject_send_event(ActiveObject_t * const dest, Event_t const e)
{

    ASSERT(dest!=0);

    cpu_irq_enter_critical();
    EventQueue_enqueue(&dest->m_eventQueue, e);
    cpu_irq_leave_critical();
}
```

- ¿Cómo se recibe un evento?

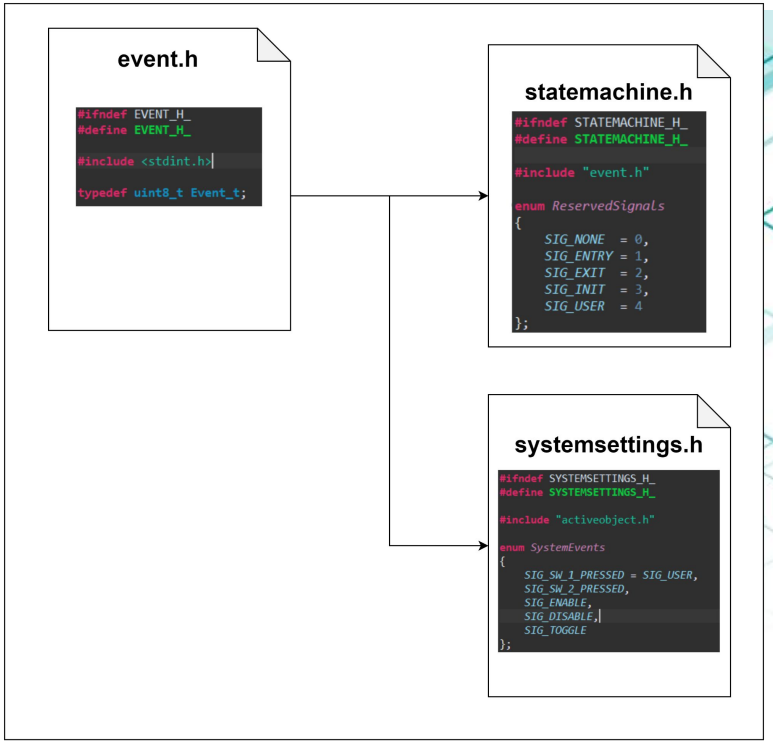
```c
Event_t ActiveObject_get_event(ActiveObject_t * const me)
{
    Event_t aux;

    ASSERT(me!=0);

    cpu_irq_enter_critical();
    if( EventQueue_dequeue(&me->m_eventQueue, &aux) == FALSE )
    {
        aux = SIG_NONE;
    }
    cpu_irq_leave_critical();

    return aux;
}
```

- Funciones *interrupt safe*.
- Se pueden llamar desde el main y también desde interrupciones.
- Esto último permite que lleguen eventos desde el exterior (por ejemplo, mensajes por UART, SPI, etc)

5. Implementación en C

HITEC SRL
sensing & control

- ¿Cómo se definen los eventos?
- **event.h** → definición del tipo de dato.
- **statemachine.h** → contiene eventos internos, propios de cualquier máquina de estados. Estos **nunca** deben pasarse usando `ActiveObject_send_event()`!
- **systemsettings.h** → eventos propios de la aplicación, definidos por el desarrollador. **Estos sí, son los que se intercambian entre Active Objects.**



5. Implementación en C

HITEC SRL
sensing & control

- ¿Una vez que un Active Object tiene un evento, cómo lo procesa?

- (1): Se copia el puntero a función del estado actual.
- (2): Se llama a la función del estado actual, pasándole el evento a procesar.
- (3): Hubo una transición de estado?
- (4) y (5): En ese caso hay que ejecutar las acciones de salida del estado previo y la de entrada al estado nuevo.

```c
#ifndef ACTIVEOBJECT_H_
#define ACTIVEOBJECT_H_

#include "statemachine.h"
#include "eventqueue.h"

#define ActiveObject_constructor(me_, initial_) ( StateMachine_constructor(&((me_)->m_super), initial_) )
#define ActiveObject_dispatch(me_, e_)          ( StateMachine_dispatch(&((me_)->m_super), e_) )
```

```c
void StateMachine_dispatch(StateMachine_t * const me, Event_t const e)
{
    StateHandler_t s = me->m_state;          (1)
    StateResult_t r = (*s)(me,e);            (2)

    if(r == RES_TRANSITION)                  (3)
    {
        (*s)(me, SIG_EXIT);                  (4)
        (me->m_state)(me, SIG_ENTRY);        (5)
    }
}
```

5. Implementación en C

HITEC SRL
sensing & control

- Las funciones de los estados tienen todas la misma pinta.
- **Si el evento es ignorado** → retorna STATEMACHINE_IGNORED().
- **Si el evento es procesado** → retorna STATEMACHINE_HANDLED().
- **Si ocurre una transición** → retorna STATEMACHINE_TRANSITION(me, estado_destino).

```c
static StateResult_t Application_active(Application_t * const me, Event_t const e)
{
    switch(e)
    {
        case SIG_ENTRY:
            ActiveObject_send_event(AOled1, SIG_ENABLE);
            ActiveObject_send_event(AOled2, SIG_ENABLE);
            HAL_TIM_Base_Start_IT(me->m_htim);
            return STATEMACHINE_HANDLED();

        case SIG_SW_2_PRESSED:
            //ActiveObject_send_event(AOled1, SIG_TOGGLE);
            ActiveObject_send_event(AOled2, SIG_TOGGLE);
            return STATEMACHINE_HANDLED();

        case SIG_SW_1_PRESSED:
            return STATEMACHINE_TRANSITION(me, Application_idle);

        case SIG_EXIT:
            HAL_TIM_Base_Stop(me->m_htim);
            ActiveObject_send_event(AOled1, SIG_DISABLE);
            ActiveObject_send_event(AOled2, SIG_DISABLE);
            return STATEMACHINE_HANDLED();
    }
    return STATEMACHINE_IGNORED();
}
```

```c
typedef enum
{
    RES_HANDLED    = 0,
    RES_IGNORED    = 1,
    RES_TRANSITION = 2
} StateResult_t;

#define STATEMACHINE_HANDLED() (RES_HANDLED)
#define STATEMACHINE_IGNORED() (RES_IGNORED)
#define STATEMACHINE_TRANSITION(me_,target_) (((StateMachine_t*)me_)->m_state = (StateHandler_t)(target_), RES_TRANSITION)
```

6. Herramienta: *QP Framework*

HITEC SRL
sensing & control

- https://www.state-machine.com/
- Miro Samek: desarrolló un software que:
  - Permite dibujar máquinas de estado.
  - Genera el código automáticamente a partir del dibujo.
- Desarrolló un framework que hace funcionar la comunicación entre *Active Objects*.
- Hay versiones con y sin RTOS, para distintas arquitecturas.
- Tiene muchas más funcionalidades…

Si bien es Open Source, se necesita una licencia para uso comercial…

7. Resumen

HITEC SRL
sensing & control

- Buscamos tener un **código entendible**, con el objetivo de facilitar el agregado de funcionalidades, las modificaciones, la detección y corrección de bugs y otros problemas.
- Las máquinas de estado permiten **implementar lógicas complejas de una manera entendible**, a través de un diagrama.
- La separación en *Active Objects* favorece la **modularización**, tanto del sistema como del código.
- El hecho de tener un framework permite que uno pueda concentrarse en **diseñar** los *Active Objects* a nivel sistema, ya que su implementación y la comunicación entre estos está resuelta.

8. Trabajo Futuro

HITEC SRL
sensing & control

- Implementar máquinas de estado jerárquicas.
- Que los eventos no sean solo flags sino que tengan datos como mensajes de tamaño variable (memoria dinámica o memory pools).
- Incluir un sistema que administre eventos de tiempos, usando un solo timer.

HITEC SRL
sensing & control

¡Gracias!

¿Preguntas?